

libquantum (SPEC CINT2006 462.libquantum) 测试报告

一、测试概述

1.1 程序来源

项目	详情
基准测试	SPEC CPU2006 CINT2006 — 462.libquantum
源码版本	libquantum 1.1.1
算法	Shor's Quantum Factoring Algorithm（秀尔量子分解算法）
输入	N=15（test input set）
预期输出	$15 = 3 \times 5$

1.2 测试意义

462.libquantum 是 SPEC CINT2006 整数组件的典型代表，模拟量子计算机的量子门操作。该程序具有以下流水线行为特征：

行为特征	来源
大量整数运算	量子态展开、模幂运算、欧几里得算法
复数运算（浮点）	量子门操作（Hadamard、CNOT、Toffoli 等）涉及复数乘法
密集内存访问	量子态向量（amplitude/state 数组）频繁读写
大量分支	量子测量、条件判断、概率分支
长依赖链	量子态演化步骤间存在数据依赖
除法等长延迟操作	分数逼近（quantum_frac_approx）、模逆运算

1.3 运行环境

项目	配置
仿真器	gem5 25.1.0.0 (RISCV build)
模拟模式	Syscall Emulation (SE)
CPU 模型	RiscvO3CPU (OpenC910-like O3)
主频	1 GHz
内存	512 MiB DDR3-1600
I-Cache	64 KiB, 2-way

项目	配置
D-Cache	64 KiB, 2-way
ROB	64 entries
IQ 类型	8 种 (AIQ0/AIQ1/BIQ/LSIQ/SDIQ/VIQ0/VIQ1/VMB)

1.4 文件清单

文件	用途
tests/test-progs/libquantum-1.1.1/	libquantum 1.1.1 源码（交叉编译）
tests/test-progs/c910-libquantum/bin/riscv/linux/shor	RISC-V 静态二进制 (649 KB)
configs/example/c910_libquantum_se.py	gem5 仿真配置脚本
tests/c910/analyze_libquantum_stats.py	统计结果验证脚本
tests/c910/run_libquantum.sh	一键运行脚本

二、运行结果

2.1 算法正确性

```
N = 15, 22 qubits required
Random seed: 8
Measured 128 (0.500000), fractional approximation is 1/2.
Possible period is 2.
15 = 3 * 5
```

Shor's 算法正确分解 15 = 3 × 5

2.2 性能指标

指标	值	说明
仿真退出 tick	5,112,590,000	~5.1 ms 仿真时间
总周期	5,112,592	pipeline cycles
提交指令数	7,860,781	~7.86M 条
执行指令数	7,959,779	~7.96M 条
IPC	1.5375	每周期提交 1.54 条指令
CPI	0.650	每条指令 0.65 周期

2.3 统计数据验证

#	检查项	结果	详情
0	Shor 算法正确性	PASS	15 = 3 × 5
1	提交指令数 > 0	PASS	7,860,781
2	IPC > 0	PASS	1.5375
3	8 条 Pipe 全部活跃	PASS	8/8
4	ROB_FULL 发生	PASS	718,690 cycles (14.1%)
5	DIV_STALL 发生	PASS	22 cycles
6	EX1 forward > 0	PASS	7,949,773
7	EX2 writeback > 0	PASS	47,527
8	IQ reads > 0	PASS	21,086,979
9	IQ writes > 0	PASS	8,305,809
10	Scoreboard wake > 0	PASS	5,397,737
11	IQ add > 0	PASS	8,077,242
12	访存指令 > 0	PASS	1,968,803

结果：12/12 checks passed — 全部通过

三、流水线详细分析

3.1 8 条 Pipe 分布

Pipe	类型	指令数	占比	对应量子操作
Pipe0	IU_ALU	3,605,766	45.5%	量子态运算、经典整数运算
Pipe1	IU_ALU_MLA	1,695	0.02%	矩阵乘法中的累加
Pipe2	BJU	2,313,508	29.2%	量子门条件分支、测量判断
Pipe3	LSU_LOAD	1,357,560	17.1%	量子态振幅读取
Pipe4	LSU_SPECIAL	145	<0.01%	FENCE/barrier 同步
Pipe5	LSU_STORE	635,369	8.0%	量子态振幅写入
Pipe6	VFPU_ALU	27,773	0.35%	复数运算 (sin/cos/exp)
Pipe7	VFPU_MA	17,963	0.23%	浮点乘加 (复数乘法)

分析：libquantum 以整数 ALU + 分支 + Load 为主，浮点占比较小但不可忽略（量子门的复数运算），8 条 Pipe 全部参与运算。

3.2 Stall 分布

Stall 类型	周期数	占比	说明
no_stall	4,372,892	85.5%	正常流水线运行
ROB_FULL	718,690	14.1%	ROB 满，rename 阻塞
DIV_STALL	22	<0.01%	整数除法单元忙

分析：主要瓶颈是 ROB_FULL（64 entries），在大量独立量子态运算时 ROB 很快填满。DIV_STALL 极少（仅 22 周期），因为 libquantum 的除法操作较少。

3.3 EX1 vs EX2 路径分离

路径	指令数	占比	延迟	典型操作
EX1	7,949,773	99.4%	1 cycle	IntAlu, Branch, Load/Store
EX2	47,527	0.6%	2-5 cycles	IntMult(3c), IntDiv(20c), FP_ADD(3c), FP_MULT(5c)

分析：libquantum 以单周期指令为主，EX2 路径占比小但存在，验证了多周期功能单元的正确调度。

3.4 功能单元忙状态

功能单元	忙周期数	说明
IntAlu	1	几乎无争用
IntMult	16	乘法操作较少
IntDiv	24	除法操作（非流水线，20 周期延迟）
MemRead	126	内存读操作争用
MemWrite	493	内存写操作争用最多
FloatMult	12	复数乘法
FloatDiv	7	复数除法
FloatMemRead	35	浮点加载争用
FloatMemWrite	11	浮点存储争用

分析：MemWrite 争用最多（493 次），说明 store buffer 和 LSQ 存在排队。IntDiv 忙 24 次对应 stall 的 22 周期。

3.5 IQ 类型活动

操作	整数 IQ	浮点 IQ	向量 IQ
Reads	20,934,621	152,358	0
Writes	8,228,396	77,413	0

分析：整数 IQ 占据绝对主导，浮点 IQ 有少量活动（复数运算），向量 IQ 无活动（SE 模式不支持 RVV）。

3.6 缓存性能

指标	值
D-Cache 命中	1,934,419
D-Cache 未命中	9,152
总访问	1,943,571
命中率	99.53%
未命中率	0.47%
平均未命中延迟	53,408 cycles

分析：libquantum 的量子态向量大小适中（22 qubits = 2^22 = 4M entries），但实际稀疏表示只使用少量非零项，64KB cache 足以容纳活跃工作集。

3.7 访存指令分布

类型	指令数	占比
Load	1,339,334	68.0%
Store	629,469	32.0%
合计	1,968,803	100%

Load/Store 比例约 2.1:1，符合量子态读写模式（读取旧状态 + 写入新状态）。

3.8 流水线状态

状态	周期数	占比
IQ 空	4,561,725	89.2%
Pipeline 空	4,554,321	89.1%
Pipeline 满	718,712	14.1% (ROB_FULL)
活跃周期	558,271	10.9%

分析：libquantum 有较多 pipeline empty 周期，这是因为量子门操作之间有同步点（barrier），导致流水线需要排空后重新填充。在活跃期间，由于 ROB 限制（64 entries），IPC 被限制在 1.54。

四、与手写微基准对比

4.1 关键指标对比

指标	c910-full-test	libquantum (SPEC 462)
提交指令数	2,660,000+	7,860,781
IPC	未统计	1.5375

指标	c910-full-test	libquantum (SPEC 462)
Pipe 活跃数	8/8	8/8
主要 Stall 类型	DIV_STALL, ROB_FULL	ROB_FULL (14.1%)
EX1/EX2 比例	6.6:1	167:1
Cache 命中率	未统计	99.53%
浮点活动	少量	27,773 VFPU ALU
分支密度	低	高 (29.2% BJU)

4.2 差异分析

方面	微基准	libquantum
代码来源	手写，刻意覆盖所有路径	真实开源项目，自然代码模式
分支密度	低（刻意简单分支）	高（29.2%，大量条件判断）
浮点使用	仅少量验证性 FP	复数运算需要大量 FP
内存模式	顺序为主	随机访问量子态向量
Stall 特征	DIV_STALL 为主	ROB_FULL 为主
EX2 占比	较高（刻意触发长延迟）	较低（自然程序以单周期为主）

4.3 互补性

微基准验证	libquantum 验证
所有 IQ 类型的刻意路由	自然代码下的 IQ 分布
IQ 溢出与 fallback	真实工作负载下无 IQ_FULL
DIV_STALL 刻意触发	少量自然除法争用
刻意依赖链（fan-out/fan-in）	算法天然依赖链
所有 Stall 类型覆盖	真实瓶颈（ROB_FULL 为主）

结论：libquantum 作为真实程序，验证了微基准无法覆盖的方面——自然分支预测、实际内存访问模式、真实依赖关系图。两者结合提供了更全面的 IEW 阶段正确性验证。

五、使用方法

```
# 一键运行（构建 + 仿真 + 分析）
./tests/c910/run_libquantum.sh

# 仅运行仿真
build/RISCV/gem5.opt configs/example/c910_libquantum_se.py
```

```
# 仅分析已有统计
python3 tests/c910/analyze_libquantum_stats.py m5out/stats.txt

# 开启调试日志
build/RISCV/gem5.opt --debug-flags=IQ,IEW,Scoreboard,Exec \
    configs/example/c910_libquantum_se.py
```

六、结论

1. **算法正确性**: libquantum 的 Shor's 算法在 C910-like O3 CPU 上正确运行，成功分解 $15 = 3 \times 5$
2. **流水线验证**: 8/8 条 Pipe 全部活跃，EX1/EX2 路径分离正确，Scoreboard 唤醒机制正常
3. **性能表现**: IPC = 1.5375，Cache 命中率 99.53%，主要瓶颈为 ROB_FULL (14.1%)
4. **统计数据验证**: 12/12 项检查全部通过，无异常
5. **与微基准互补**: libquantum 提供了真实程序的行为特征，与手写微基准形成互补验证